

Take-Home Assignment: Second-Resolution Equity Simulator

Two days, then a live demo and a discussion. Core in C++17 or later. AI tools are welcome.

1. What you build

- A **data generator** that produces one quarter of synthetic one-second prices for 2,000 stocks: 1,000 in the US, 500 in Europe and 500 in Asia (Japan).
- A **storage layer** that ingests those files into an on-disk format of your own design.
- A **simulator** that prices positions, takes market snapshots, and runs a mean-reversion strategy with parameters we choose during the demo.
- A **dashboard** that records every simulation run with its parameters, runtime, peak memory and results.
- A complete system with modest speed is worth more to us than one fast component. Please get everything working end to end on a small date range first, then improve what is slow.

2. Rules

- Storage layer and simulator in C++. Generator and helper scripts in Python or C++.
- Supporting libraries are fine: compression, threading, CLI parsing, NumPy. Please do not use a ready-made database or time-series store (kdb, DuckDB, ClickHouse, Arrow, Parquet). Designing the data layout is the point.
- The system should run on a machine with 16 GB of RAM. The full quarter needs roughly 100 GB of free disk, so develop on a shorter date range (the generator accepts one) and run the full quarter only when you are ready.

3. Data specification

You write the generator, and the output must follow this specification so that results can be compared across candidates.

- **Instruments (2,000 total):** 1,000 US, IDs 0–999, trading 09:30–16:00 New York time. 500 Europe, IDs 1000–1499, 09:00–17:30 Berlin time. 500 Asia (Japan), IDs 1500–1999, 09:00–11:30 and 12:30–15:00 Tokyo time. One price per second of each session.
- **Period:** 2 January to 31 March 2025, weekdays. US clocks change on 9 March, European clocks on 30 March, Japan has no DST. Treat these as the only holidays: US 9 January, 20 January, 17 February; Japan 13 January, 11 February, 24 February, 20 March; Europe none. All timestamps are UTC.
- **Segment:** a continuous block of trading (a US day, a European day, a Japanese morning, a Japanese afternoon). Several rules below reset at segment boundaries.
- **Files:** one per instrument per day at `raw/YYYYMMDD/NNNN.bin`. Little-endian records of 16 bytes: int64 timestamp in ns since epoch, float32 price, int32 volume. Sorted by time. Each second is dropped independently with probability 0.5% (no trade); dropped seconds are absent.
- **Per instrument, draw once:** start price uniform 10–500; annual volatility uniform 15–60%; base volume uniform 50–2,000; a 30% chance per region of being a **reverter**. Per-second volatility = annual volatility / $\sqrt{(252 \times 23,400)}$, same in all regions.
- **Price within a segment** (log price) = random walk + mean-reverting component. The walk moves each second by per-second volatility $\times$ a standard normal. The component is zero for non-reverters. For reverters it is an Ornstein–Uhlenbeck process: each second multiply by $e^{-1/\tau}$ and add a normal shock, with τ = half-life / $\ln 2$, half-life uniform 300–1,200 s (once per instrument), shock size chosen so the long-run standard deviation is uniform 0.05–0.15% (once per instrument). **The component resets to zero at every segment start.**
- **Between segments** the walk jumps: overnight with standard deviation annual volatility / $\sqrt{252 \times 0.5}$; across the Tokyo lunch with per-second volatility $\times \sqrt{3,600}$.
- **Volume** per second is Poisson with mean base volume $\times (0.5 + 2(2u - 1)^2)$, u = fraction of segment elapsed.
- **Why reverters:** a correct simulator running the strategy in 5.3 shows clearly positive Sharpe on reverters and roughly zero on the rest. Because the signal resets each segment, a simulator that ignores session boundaries shows a distorted result. That is deliberate.
- **Command:** `./gen --out raw/ --seed 20250101 --start 2025-01-02 --end 2025-03-31`. Use seed 20250101 for the demo; same seed must give identical files. Also write reverter IDs, one per line, to `raw/reverters.txt`. The quarter is about three billion points, so a per-second Python loop takes many hours; vectorise by instrument-day.

4. Storage layer

- `./ingest --raw raw/ --store store/` reads the raw directory into your format.

- Raw files are grouped by instrument and day; the simulator mostly needs many instruments at one instant on a single UTC clock across three regions. How you resolve that is the central design decision.
- Queries must work from a cold disk cache. Precomputation during ingest is fine but counts toward ingest time and store size.

5. Simulator

Timestamps are UTC ns. “Last known price at t ” means the most recent record at or before t , across segment and day boundaries.

5.1 Pricing positions

- Input CSV: `trade_id, instrument_id, side, quantity, entry_ts_ns, exit_ts_ns`, side is +1 or -1.
 - Entry snaps **forward** to the first recorded second at or after it. Exit snaps **backward** to the last recorded second at or before it. If `exit < entry` after snapping, or either cannot be resolved, status 1 and empty numbers. Positions carry across nights, weekends and lunch.
 - `pnl = side × quantity × (exit price – entry price)`. `max_adverse_excursion` = largest unrealised loss at any recorded second in `[entry, exit]`, floored at 0. This requires scanning the holding period, not two lookups.
 - Output CSV: `trade_id, status, entry_ts_ns, exit_ts_ns, entry_price, exit_price, pnl, max_adverse_excursion`. We test with one million random positions, holding one second to several days.
- ```
./sim trades --store store/ --in trades.csv --out results.csv
```

### 5.2 Snapshots

- Input: one UTC timestamp per line. Output: binary float32[N][2000], row  $i$  = last known price of every instrument at timestamp  $i$ , NaN if none.
  - Timestamps are random over the quarter including nights and weekends. We record median and p99 latency per snapshot over 10,000 queries.
- ```
./sim snapshot --store store/ --in timestamps.txt --out snapshots.bin
```

5.3 Mean-reversion strategy (the main demo workload; parameters must not be hard-coded)

- Parameters: window (seconds), entry threshold, exit threshold, max hold (seconds). Runs independently per instrument. No transaction costs.
 - At each recorded second in a segment, once a full window of recorded seconds has elapsed in that segment, compute mean and population standard deviation of price over the trailing window and $z = (\text{price} - \text{mean}) / \text{std}$. If `std = 0`, no signal.
 - No position: $z > \text{entry}$ opens a short; $z < -\text{entry}$ opens a long. `Quantity = floor(10,000 / price)`.
 - Open position: close when z reaches or crosses the exit threshold (long: $z \geq \text{exit}$; short: $z \leq -\text{exit}$), or when max hold has passed, or at the last recorded second of the segment, whichever is first. One position per instrument at a time; no re-entry in the exit second.
 - Output trades CSV:
`instrument_id, side, quantity, entry_ts_ns, exit_ts_ns, entry_price, exit_price, pnl`. Output summary CSV: `region, reverter_flag, trade_count, total_pnl, sharpe`, six rows. Sharpe = `mean / sample std of daily pnl × √252`, where daily pnl sums trades exited on that local trading day.
- ```
./sim strategy --store store/ --reverters raw/reverters.txt --window 1800 --entry 2.0 --exit 0.0 \
--max-hold 3600 --out strategy_trades.csv --summary summary.csv
```

## 6. Dashboard

- Records every simulation run: parameters, wall-clock time, peak memory, and the six-row summary. A local web page is ideal; a terminal view is acceptable.
- Must update on its own when a run finishes. We will watch it during the demo. Polish does not matter; visibility of runtime and memory does.

## 7. Demo flow

- Run the generator on a small date range, then ingest, so we see the pipeline from the start.
- Run the strategy on the full quarter with parameters of your choosing; walk us through reverters vs non-reverters by region on the dashboard.
- We suggest two or three parameter sets on the spot and watch runtime and memory as they run.
- Show how you checked your own correctness: for example, a small hand-verifiable dataset, or tests for the snapping rules and segment boundaries.
- Discussion questions will be along the lines of: 20,000 instruments? Add Hong Kong with its lunch break? Where is the snapshot p99 coming from? What breaks at millisecond resolution?

## 8. What we measure

- Generation time for the quarter. Ingest time and store size in bytes per point (raw is 16).
- One million positions: time, cold and warm cache. 10,000 snapshots: median and p99 latency.
- Each strategy run: time and peak memory, as shown on your dashboard. Correctness of results, and how you verified it.
- As a point of reference, a plain implementation (file per instrument, timestamps stored, binary search, single thread) is the kind of solution we expect to be improved on; comparing your numbers against your own simple first version is a good way to show progress.
- Roughly half the assessment is these numbers; half is the design discussion: clarity of your layout and why it fits, what you measured, honesty about shortcuts, and how you reason about new questions.
- Extra credit for clearly highlighting the challenges you hit and how you solved them.
- Creative solutions earn more points. If you found an unusual way to make something faster, smaller or simpler, please show it.

## 9. Deliverables

- **Code:** a repository link or zip with all source, a build script and a short README, sent by the end of day two.
- **Onsite presentation:** no more than 15 slides, covering the challenges you ran into, the ideas and innovations you brought to the design, and the details you think we should know. This replaces a written design document.
- Where this document leaves something open, please make a reasonable assumption, note it in your README, and carry on.
- Thank you, and we look forward to seeing what you build.